Arjun Powerpack RUL
XGBoost + Kalman on a Synthetic Benchmark

Predictive maintenance for an Arjun-like tank powerpack: problem, architecture, methods, results, deployment reality.

Metric (full test, 143,904 rows)	Production best
MAE (h)	1.723
RMSE (h)	2.315
MSE	5.358
R2	86.27%
Within 1 h	41.30%
Within 2 h	68.58%
Package	1.89 MiB UBJ
Edge latency*	~0.017 s / 360 rows

** Dev-machine timing, not ECU. HIL measurement still required. Results are exploratory: the test cohort was inspected across runs; a fresh frozen cohort is needed for a final claim. No classified or official Arjun data is used anywhere.*

Date: 13 Oct 2026. Code: Predictive_maintenance repo. Production path: observed telemetry -> causal despike -> causal features -> train-fitted preprocessing -> XGBoost (UBJ) -> scalar Kalman -> linear calibration -> RUL(hours).

1. Problem statement

The Arjun Mk-1 powerpack (MTU MB 838 Ka-501 V10, ~1400 hp at 2400 rpm, Renk gearbox, 58.5 t tank) must run ~600 h / 5000 km between overhauls. Documented pain: engine-transmission mismatch (6 transmission failures before 6000 km), transmission-oil overheating, winter-trial engine failures, and a CAG-reported overhaul backlog (9 powerpacks awaiting repair). A causal remaining-useful-life (RUL) estimate from on-board sensors would let the Army pull a powerpack for service before it fails in the field.

Task: from 1-minute telemetry of 8 powerpack channels, predict RUL in hours for each timestamp, using only present/past data (causal, edge-deployable). Label: RUL = (run-to-failure time - now). The benchmark is fully synthetic (docs/public_sensor_evidence.md): public channel types and platform anchors are real; every numeric history is simulated with operating regimes, environments, degradation modes, missing blocks and spikes.

Why this is hard

- Degradation signal is tiny early in life (health 0.03-0.07) vs load swings (demand 0.05-0.88), ambient spread (coolant means 77-103 C by environment) and sensor noise.
- 53% of test MSE comes from 24% of rows (RUL 15-30 h, early life): RMSE 3.41 there vs 0.94 at RUL 0-2 h.
- Test cohort is harder than validation: transmission_wear 31 vs 14 engines, lifetime std 3.42 vs 2.85 h.

2. Why XGBoost + Kalman (and why LSTM is only a baseline)

- XGBoost: tabular, compact (1.96 MB booster), deterministic, compiles to dependency-free C (treelite/timber), no GPU, no recurrent runtime. Fits ECU flash (2-16 MB) and real-time budgets.
- Kalman: 2-state [RUL, dRUL/dt] causal smoother. Enforces physically plausible decline (RUL falls ~1 min/min, never negative, velocity never positive). ~4 KB state+code, big NASA-score cut on spiky data.
- LSTM baseline: 60-step sequence model. More accurate here (common test 1.581/2.228/86.19% vs XGB+Kalman 1.685/2.291/85.40%) because it sees raw temporal context, but needs a recurrent runtime, 60-min warm-up, 5-min cadence eval, and GPU training. Rejected for the tank ECU; kept as the accuracy reference.
- What was tried and rejected: per-slice specialist mixtures (lose to single model - more data wins), isotonic calibration (overfits across engines), pseudohuber (collapses), lifetime-direct prediction (2.74 vs 2.26 RMSE).

3. Data and splits

- 1,200 engines (800 original + 400 extra train-only, same seed stream), 1,408,077 rows, 1-min cadence, 720-1600 rows/engine.
- 8 sensors: engine/transmission rpm, coolant + trans-oil temp, oil + clutch pressure, solenoid current, vehicle speed.
- Regimes: idle/cruise/stop-go/acceleration/hill-climb/rough-terrain. Envs: temperate/hot-desert/cold/high-altitude. Failure modes (latent, never features): thermal/lubrication/transmission/mixed.
- Faults: ~2.3% missing (0.15% IID + 1-4 burst gaps of 4-17), 0.07% spikes.
- Split (chronological cohorts): train 960 (1-560 + 801-1200), val 120 (561-680), test 120 (681-800). Engine-disjoint. Selection on val only; test evaluated after freeze.

4. Architecture and flow

telemetry -> causal median-5 despike -> causal features (lags 1/15, windows 15/60, EWM 120/360, physics residuals, operating-hours) -> train-fitted quantile clip + median impute -> XGBoost (squarederror, early-life weighted) -> scalar Kalman (RMSE-selected) -> linear calibration -> RUL. Train path mirrors it with a dev/val split, then a final refit on train+val and one frozen test evaluation. Key files: src/features.py, src/preprocess.py, src/kalman.py, src/train_pipeline.py, scripts/final_push3.py, src/predict_edge.py, src/benchmark_edge.py.

Causal feature highlights (129 total)

- Per sensor: missing flag, lag1/15, mean/std/slope over 15/60, EWM120/360 + deltas.
- Physics residuals vs load proxy (rpm/2400): rpm gap, coolant/trans-oil deltas, pressure-per-krpm, slip proxy, temp excess vs load, pressure deficits, clutch/solenoid residuals, accel + rate terms.
- Operating hours (service_hours_since_overhaul): allowed - known at inference from tank logs, past-only, 1 float. timestamp_min stays banned as its duplicate; env/failure/latent/future stay banned.
- Causal median-5 despike before features (past 5 only, per engine; thresholds e.g. temps 8 C, rpm 120, oil pressure 0.8 bar).

Training specifics

- Objective reg:squarederror (RMSE target; absolute-error optimizes the wrong metric here). Sample weight RUL>10 h x2 (early rows dominate MSE). Depth 6, lr 0.03, 400 trees, min_child_weight 12, lambda 10, colsample 0.7, subsample 0.8, max_bin 128.
- Preprocessing (quantile 0.005-0.995 clip + medians) fit on train only at selection, refit on final sample (324k rows from 1080 engines).
- Kalman grid of 8 (q_pos/q_vel/R) scored by val RMSE; winner (0.01, 1e-05, 0.60). Scalar 1-D update (no matrix inverse): 44x faster, identical math; first output equals raw measurement so offline and online paths agree.
- Linear calibration scale 1.02 / offset +0.10 h, RMSE-selected on val tracked predictions (same protocol as the LSTM cal).

5. Results

Full held-out test (143,904 rows)

Model	MAE	RMSE	MSE	R2	w1h
XGB raw	1.758	2.382	5.672	85.47%	41.55%
+Kalman	1.746	2.369	5.613	85.62%	41.69%
+Kalman+cal	1.723	2.315	5.358	86.27%	41.30%

Vs previous production (1.750/2.407/5.793/85.16%): RMSE -3.8%, MSE -7.5%, R2 +1.11pp. Dev val behind it: raw 2.146 -> Kalman 2.129 -> cal 2.101 / 87.92%.

Common XGB/LSTM cadence (27,416 rows)

Model	MAE	RMSE	R2	w1h
XGB	1.699	2.306	85.21%	42.50%
XGB+Kal	1.685	2.291	85.40%	42.66%
LSTM cal	1.581	2.228	86.19%	48.29%

Where error lives (production hybrid slices)

Worst slice	MAE	RMSE	R2
transmission_wear	2.057	2.717	80.45%
high_altitude	2.006	2.770	79.50%
thermal_aging	1.962	2.591	82.35%
lube_degrad. (best)	1.583	2.113	86.14%

Loop record: 2.407 -> 2.338 -> 2.315 test RMSE across pushes. Even a cheating oracle gate only reached 2.12 val: early-life SNR is the wall, not tuning effort.

6. Deployment constraints (Arjun ECU reality)

- SWaP-C: flash 2-16 MB, RAM KBs-low MBs. Package here 1.89 MiB UBJ (booster 1.87 MB); JSON form was 3.97 MB - binary UBJ is the deploy format. Next squeeze: quantization to INT8 (~4x), pruning, top-80 feature cut.
- No Python runtime on ECU (200 MB+ RSS): ship compiled C via treelite/timber; features + scalar Kalman are a few hundred ops/row, fully deterministic.
- Latency (dev box, 360-row history): feat 0.0116 s + XGB 0.0026 s + Kalman 0.0032 s = ~0.017 s. ECU will differ - HIL p50/p95/p99, peak RAM, cold start, 72-h soak still required.
- Env: MIL-STD-810 (heat/vibration/dust) + MIL-STD-461 (EMC); CAN/MIL-STD-1553 sensor bus; offline-first (no connectivity); sensor faults (frozen/drift/dropout) must map to quality flags + abstention, not silent bad RUL.
- Model risks: test cohort inspected 3x (exploratory, not one-shot); DATRAN-1500 engine change will shift degradation signatures (plan retrain/transfer); no uncertainty intervals yet (quantile heads are the honest next step).

7. Reproduce in 6 commands

```
pip install -r requirements.txt
python src/simulate_dataset.py --config config/config.yaml
python src/validate_dataset.py --config config/config.yaml
python -m pytest -q
python src/train_pipeline.py --config config/config.yaml   # canonical path
python src/evaluate.py --config config/config.yaml && python src/benchmark_edge.py
```

8. Likely interview questions (30-second answers)

- Q: Why not LSTM in production? A: Best accuracy, but recurrent runtime + warm-up + size/latency vs a 1.9 MB deterministic tree+filter that compiles to C.
- Q: Is operating-hours a leak? A: No - logged on the tank, past-only. Env/failure/latent/future stay banned with unit tests.
- Q: Why squarederror not MAE? A: The goal is RMSE/R2; MAE objective optimizes the wrong loss (measured: 2.295 vs 2.232).
- Q: Kalman value-add? A: Jitter/spike smoothing + physical monotonicity; small RMSE gain, large NASA-score cut; scalar form is 44x faster.
- Q: Why not R2 90? A: Early-life physics: signal 0.03-0.07 health vs 26 C ambient spread. Needs stronger sensors (vibration/debris), not more tuning.
- Q: What is missing for the field? A: Real telemetry, frozen-cohort claim, HIL, uncertainty/OOD, safety case, DATRAN retrain plan.